

Lecture 7 – Function (Part 2)

DITG 1113 COMPUTER PROGRAMMING

LEARNING OUTCOMES

- ✖ At the end of this lecture, you should be able to
 - + use reference variables as parameters
 - + when to use function that pass by value and pass by reference

USING REFERENCE VARIABLES AS PARAMETERS

- ✖ A mechanism that allows a function to work with the original argument from the function call, not a copy of the argument
- ✖ Allows the function to modify values stored in the calling environment
- ✖ Provides a way for the function to ‘return’ more than one value

PASSING BY REFERENCE

- ✗ A **reference variable** is an alias for another variable
- ✗ Defined with an ampersand (&)

```
void getDimensions(int &, int &);
```
- ✗ Changes to a reference variable are made to the variable it refers to
- ✗ Use reference variables to implement passing parameters *by reference*

Program 6-25

The & here in the prototype indicates that the parameter is a reference variable.

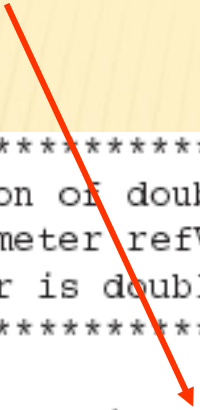
```
1  // This program uses a reference variable as a function
2  // parameter.
3  #include <iostream>
4  using namespace std;
5
6  // Function prototype. The parameter is a reference variable.
7  void doubleNum(int &);
8
9  int main()
10 {
11     int value = 4;
12
13     cout << "In main, value is " << value << endl;
14     cout << "Now calling doubleNum..." << endl;
15     doubleNum(value);
16     cout << "Now back in main. value is " << value << endl;
17     return 0;
18 }
19
```

Here we are passing the value by reference.

(Program Continues)

Program 6-25 (Continued)

The & also appears here in the function header.



```
20  /*******  
21  // Definition of doubleNum. *  
22  // The parameter refVar is a reference variable. The value *  
23  // in refVar is doubled. *  
24  /*******  
25  
26  void doubleNum (int &refVar)  
27  {  
28      refVar *= 2;  
29  }
```

Program Output

```
In main, value is 4  
Now calling doubleNum...  
Now back in main. value is 8
```

REFERENCE VARIABLE NOTES

- ✗ Each reference parameter must contain &
- ✗ Space between type and & is unimportant
- ✗ Must use & in both prototype and header
- ✗ Argument passed to reference parameter must be a variable – cannot be an expression or constant
- ✗ Use when appropriate – don't use when argument should not be changed by function, or if function needs to return only 1 value

WHAT IS THE DIFFERENCE BETWEEN PASSING BY VALUE AND PASSING BY REFERENCE



Best explanation I ever heard of this.

121



Say I want to share a web page with you.

If I tell you the URL, I'm **passing by reference**. You can use that URL to see the **same web page** I can see. If that page is changed, we both see the changes. If you delete the URL, all you're doing is destroying your reference to that page - you're not deleting the actual page itself.

If I print out the page and give you the printout, I'm **passing by value**. Your page is a disconnected copy of the original. You won't see any subsequent changes, and any changes that you make (e.g. scribbling on your printout) will not show up on the original page. If you destroy the printout, you have actually destroyed **your copy** of the object - but the original web page remains intact.

share | improve this answer

answered Jan 10 '09 at 13:28



Dylan Beattie

18k ● 10 ● 51 ● 110

Source: <http://stackoverflow.com/questions/373419/whats-the-difference-between-passing-by-reference-vs-passing-by-value>

C++ STANDARD LIBRARY

- ✗ The C++ run-time library provides several functions for performing specific operations such as performing complex mathematical operations.
- ✗ The C++ Standard Library can be categorized into two parts:
 - + **The Standard Function Library:** This library consists of general-purpose, stand-alone functions that are not part of any class. The function library is inherited from C.
 - + **The Object Oriented Class Library:** This is a collection of classes and associated functions.
- ✗ Standard C++ Library incorporates all the Standard C libraries also, with small additions and changes to support type safety.

THE STANDARD FUNCTION LIBRARY

- ✗ The standard function library is divided into the following categories:
 - + I/O
 - + String and character handling
 - + Mathematical
 - + Time, date, and localization
 - + Dynamic allocation
 - + Miscellaneous
 - + Wide-character functions

THE OBJECT ORIENTED CLASS LIBRARY

- ✖ Standard C++ Object Oriented Library defines an extensive set of classes that provide support for a number of common activities, including I/O, strings, and numeric processing. This library includes following:
 - + The Standard C++ I/O Classes
 - + The String Class
 - + The Numeric Classes
 - + The STL Container Classes
 - + The STL Algorithms
 - + The STL Function Objects
 - + The STL Iterators
 - + The STL Allocators
 - + The Localization library
 - + Exception Handling Classes
 - + Miscellaneous Support Library

THE `EXIT()` FUNCTION

- ✗ Terminates the execution of a program
- ✗ Can be called from any function
- ✗ Can pass an `int` value to operating system to indicate status of program termination
- ✗ Usually used for abnormal termination of program
- ✗ Requires `cstdlib` header file
- ✗ Use carefully

THE EXIT () FUNCTION

- ✗ Use an integer value to indicate program status
- ✗ Often, 0 means successful completion, non-zero indicates a failure condition

- ✗ Example:

```
exit (0) ;
```

- ✗ Can use the two constants define in `cstdlib` header that are commonly passed, to indicate success or failure:

```
exit (EXIT_SUCCESS) ;
```

```
exit (EXIT_FAILURE) ;
```

ASK YOURSELF

- ✖ Do you know how to use reference variables as parameters?
- ✖ Do you know why and when to use function that pass by reference?
- ✖ Do you know why and when to use function that pass by value?